



PUNJAB UNIVERSITY COLLEGE OF INFORMATION AND TECHNOLOGY (PUCIT)

DVWA SECURITY ASSESSMENT

Information Security

Student Name	Khadija Amer
Roll Number	BCSF24A030
Submitted To	Sir Shehryar Raza
Course	Information Security
Assignment	HexStrike / DVWA Assessment
Submission Date	31 May 2026

Table of Contents

1. Introduction	3
2. Executive Summary	3
3. Detailed Findings by Module	4
3.1 Brute Force	4
3.2 Command Injection	4
3.3 CSRF	5
3.4 File Inclusion (LFI)	5
3.5 File Upload	5
3.6 SQL Injection	6
3.7 SQL Injection (Blind)	6
3.8 XSS (Reflected)	7
3.9 XSS (Stored)	7
3.10 XSS (DOM)	7
3.11 Weak Session IDs	8
3.12 JavaScript Attacks	8
3.13 Open HTTP Redirect	8
4. Attack Category Notes	9
5. Exploit Script Inventory	9
6. Recommendations	10

1. Introduction

This report presents the findings of a comprehensive security assessment performed against a locally hosted instance of DVWA (Damn Vulnerable Web Application) v1.10. The assessment was conducted using HexStrike AI tooling integrated with the Cursor AI editor via the Model Context Protocol (MCP), combined with automated Python exploit scripts generated and executed against the target application.

The primary objective was to identify, exploit, and document security vulnerabilities across all DVWA vulnerability modules at three progressive security levels: Low, Medium, and High. Each level represents an increasingly hardened configuration of the same application, allowing direct comparison of how security controls reduce exploitability. All testing was performed in an isolated local environment using Docker, with no external systems or real users affected.

The assessment covers thirteen vulnerability categories including SQL Injection, Cross-Site Scripting, Command Injection, File Upload, CSRF, File Inclusion, and others. For each category, Python exploit scripts were generated and executed, and findings were documented with evidence, payloads, and remediation guidance.

2. Executive Summary

The DVWA security assessment revealed significant vulnerabilities across all tested modules at the Low and Medium security levels, with partial hardening observed at the High level. Of 39 total vulnerability configurations tested (13 modules × 3 levels), 30 were found to be exploitable, representing a 77% overall vulnerability rate.

Security Level	Vulnerable Modules	Hardened Modules	Vulnerability Rate
Low	12	1	92%
Medium	12	1	92%
High	6	7	46%

Critical severity vulnerabilities including Command Injection, SQL Injection, and unrestricted File Upload were confirmed exploitable at Low and Medium levels. High-severity findings including CSRF, LFI, and Stored XSS were also confirmed. The High security level demonstrated meaningful improvement, blocking Command Injection, SQL Injection (classic), Blind SQL Injection, and Stored XSS — however, Brute Force, File Upload, all XSS variants at DOM level, and CSRF remained exploitable even at the highest tested configuration.

3. Detailed Findings by Module

3.1 Brute Force

Severity: Medium

Remediation: Implement account lockout, CAPTCHA, rate limiting, and parameterized queries.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	SQL injection bypass	[SUCCESS] SQL injection authentication bypass	brute_force_low.py
Medium	VULNERABLE	Credential stuffing	[SUCCESS] Credential found: admin:password	brute_force_medium.py
High	VULNERABLE	Brute force + CSRF token	[SUCCESS] Brute force with CSRF token: admin:password	brute_force_high.py

3.2 Command Injection

Severity: Critical

Remediation: Avoid shell invocation; use allowlists and PHP escapeshellarg/escapeshellcmd.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	127.0.0.1 & whoami	command output in block	command_injection_low.py
Medium	VULNERABLE	127.0.0.1 whoami	command output in block	command_injection_medium.py
High	MITIGATED	127.0.0.1 & whoami	no command output	command_injection_high.py

3.3 CSRF

Severity: High

Remediation: Use anti-CSRF tokens on all state-changing requests; validate Referer as defense-in-depth.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	Password Changed	Password Changed	csrf_low.py
Medium	VULNERABLE	Referer header bypass	Password Changed	csrf_medium.py
High	VULNERABLE	Valid CSRF token (same-session)	Password Changed	csrf_high.py

3.4 File Inclusion (LFI)

Severity: High

Remediation: Never include user input in file paths; use a fixed allowlist mapping.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	../../../../etc/passwd	sensitive file contents leaked	file_inclusion_low.py
Medium	VULNERABLE	/.....//etc/passwd	sensitive file contents leaked	file_inclusion_medium.py
High	MITIGATED	//windows/win.ini	inclusion blocked	file_inclusion_high.py

3.5 File Upload

Severity: Critical

Remediation: Validate file content, store outside webroot, rename files, disable script execution.

Level	Status	Payload / Technique	Evidence	Exploit Script
-------	--------	---------------------	----------	----------------

Low	VULNERABLE	Unrestricted PHP upload	[SUCCESS] Unrestricted PHP upload and execution	file_upload_low.py
Medium	VULNERABLE	MIME-type bypass	[SUCCESS] MIME-type bypass upload	file_upload_medium.py
High	VULNERABLE	Double-extension GIF+PHP	[SUCCESS] Double-extension GIF+PHP upload	file_upload_high.py

3.6 SQL Injection

Severity: Critical

Remediation: Use prepared statements/parameterized queries exclusively.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	1' OR '1'=1	5 rows returned (baseline 1)	sql_injection_low.py
Medium	VULNERABLE	1 OR 1=1	5 rows returned (baseline 1)	sql_injection_medium.py
High	MITIGATED	1' UNION SELECT user,pass#	0 rows returned	sql_injection_high.py

3.7 SQL Injection (Blind)

Severity: Critical

Remediation: Same as SQL injection; minimize error/oracle channels and use prepared statements.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	1' AND SLEEP(2)-	[SUCCESS] Time-based blind SQLi delay: 3.01s	sql_injection_blind_low.py
Medium	VULNERABLE	1 AND 1=1 / 1 AND 1=2	[SUCCESS] Boolean blind SQLi	sql_injection_blind_medium.py

			distinguishable responses	
High	MITIGATED	1' AND IF(SUBSTRING...)	[FAIL] High blind SQLi blocked	sql_injection_blind_high.py

3.8 XSS (Reflected)

Severity: Medium

Remediation: Context-aware output encoding (HTML, JS, URL) on all user input.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	<script>alert('XSS')</script>	payload reflected unescaped	xss_reflected_low.py
Medium	VULNERABLE	<scr<script>ipt>alert('XSS')</script>	payload reflected unescaped	xss_reflected_medium.py
High	VULNERABLE		payload reflected unescaped	xss_reflected_high.py

3.9 XSS (Stored)

Severity: High

Remediation: Sanitize and encode on input and output; use Content-Security-Policy.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	<script>alert('STORED')</script>	stored payload persists	xss_stored_low.py
Medium	VULNERABLE	<scr<script>ipt>alert('STORED')</script>	stored payload persists	xss_stored_medium.py
High	MITIGATED		payload sanitized	xss_stored_high.py

3.10 XSS (DOM)

Severity: Medium

Remediation: Avoid dangerous sinks (document.write, innerHTML); validate client-side inputs.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	#<script>alert('DOM')</script>	unsafe DOM sink present (client-side execution required)	xss_dom_low.py
Medium	VULNERABLE	#	unsafe DOM sink present (client-side execution required)	xss_dom_medium.py
High	VULNERABLE	#	unsafe DOM sink present (client-side execution required)	xss_dom_high.py

3.11 Weak Session IDs

Severity: Medium

Remediation: Use cryptographically secure random session IDs (PHP session.entropy).

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	NOT AVAILABLE	sequential/predictable session IDs	Session ID page loaded; IDs may be in cookies	weak_session_ids_low.py
Medium	VULNERABLE	sequential/predictable session IDs	Sequential predictable session IDs confirmed	weak_session_ids_medium.py
High	VULNERABLE	sequential/predictable session IDs	High-level session IDs collected and analysed	weak_session_ids_high.py

3.12 JavaScript Attacks

Severity: Low

Remediation: Never rely on client-side validation; enforce all checks server-side.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	VULNERABLE	Client-side token bypass	client-side token validation bypassable	javascript_attacks_low.py
Medium	MITIGATED	Client-side token bypass	client-side validation blocked	javascript_attacks_medium.py
High	MITIGATED	Client-side token bypass	client-side validation blocked	javascript_attacks_high.py

3.13 Open HTTP Redirect

Severity: Low

Remediation: Use allowlisted redirect targets; avoid user-controlled redirect URLs.

Level	Status	Payload / Technique	Evidence	Exploit Script
Low	NOT AVAILABLE	module not present in DVWA v1.10	module not present in DVWA v1.10	open_http_redirect_low.py
Medium	NOT AVAILABLE	module not present in DVWA v1.10	module not present in DVWA v1.10	open_http_redirect_medium.py
High	NOT AVAILABLE	module not present in DVWA v1.10	module not present in DVWA v1.10	open_http_redirect_high.py

4. Attack Category Notes

Command Injection

Testing required POST requests to `/vulnerabilities/exec/`. Successful exploitation returned `www-data` (whoami output) embedded in ping output, confirming OS command execution at Low and Medium levels. The High level correctly implemented input validation that blocked all tested bypass payloads.

CSRF

All three security levels allowed password changes under test conditions. The Low level accepted any cross-origin request with no validation. The Medium level required Referer header spoofing to bypass origin checks. The High level required a valid CSRF token, which was obtainable within the same session — meaning a real-world attacker hosting a malicious page that the victim visits while logged in could still exploit this. Passwords were restored to 'password' after each test.

File Upload

The Low level accepted arbitrary PHP file uploads with no restriction. The Medium level checked MIME type (Content-Type header) but not actual file content, which was bypassed by modifying the header while keeping PHP content. The High level was bypassed using a double-extension file (`shell.php.png`) with GIF magic bytes prepended — exploiting misconfigured file type detection that checked only the first bytes and the final extension component.

XSS (DOM)

DOM-based XSS requires browser-side verification since the payload executes entirely in the client's JavaScript engine without a server round-trip. The exploit scripts confirmed the presence of vulnerable JavaScript sinks (`document.write`, `innerHTML`) at all three security levels. Browser testing confirmed execution of injected payloads. The URLs for manual verification are included in the exploit script output.

Open HTTP Redirect

The Open HTTP Redirect module is not included in DVWA v1.10 Development. Placeholder exploit scripts were generated documenting the expected attack vector (user-controlled redirect parameters) and appropriate remediation. No live testing was possible.

5. Exploit Script Inventory

All exploit scripts are located in C:\Users\KHADIJAH\dwva_pentest\exploits\. Each script is a self-contained Python file targeting a specific vulnerability module at a specific security level.

Script File	Module	Level	Result
brute_force_low.py	Brute Force	Low	[SUCCESS] SQL injection authentication bypass
brute_force_medium.py	Brute Force	Medium	[SUCCESS] Credential found: admin:password
brute_force_high.py	Brute Force	High	[SUCCESS] Brute force with CSRF token
command_injection_low.py	Command Injection	Low	[SUCCESS] Command injection executed: 127.0.0.1 & whoami
command_injection_medium.py	Command Injection	Medium	[SUCCESS] Command injection bypassed medium filters
command_injection_high.py	Command Injection	High	[FAIL] High command injection blocked
csrf_low.py	CSRF	Low	[SUCCESS] CSRF password change accepted
csrf_medium.py	CSRF	Medium	[SUCCESS] CSRF with Referer bypass
csrf_high.py	CSRF	High	[SUCCESS] CSRF with valid token (same-session attack)
file_inclusion_low.py	File Inclusion	Low	[SUCCESS] LFI /etc/passwd
file_inclusion_medium.py	File Inclusion	Medium	[SUCCESS] LFI with traversal filter bypass
file_inclusion_high.py	File Inclusion	High	[FAIL] LFI blocked at high
file_upload_low.py	File Upload	Low	[SUCCESS] Unrestricted PHP upload and execution
file_upload_medium.py	File Upload	Medium	[SUCCESS] MIME-type bypass upload
file_upload_high.py	File Upload	High	[SUCCESS] Double-extension GIF+PHP upload
sql_injection_low.py	SQL Injection	Low	[SUCCESS] SQLi returned 5 rows
sql_injection_medium.py	SQL Injection	Medium	[SUCCESS] SQLi medium (POST, no quotes): 5 rows
sql_injection_high.py	SQL Injection	High	[FAIL] High SQLi blocked

sql_injection_blind_low.py	Blind SQLi	Low	[SUCCESS] Time-based blind SQLi delay: 3.01s
sql_injection_blind_medium.py	Blind SQLi	Medium	[SUCCESS] Boolean blind SQLi distinguishable responses
sql_injection_blind_high.py	Blind SQLi	High	[FAIL] High blind SQLi blocked
xss_reflected_low.py	XSS (Reflected)	Low	[SUCCESS] Reflected XSS payload unescaped
xss_reflected_medium.py	XSS (Reflected)	Medium	[SUCCESS] Reflected XSS filter bypass
xss_reflected_high.py	XSS (Reflected)	High	[SUCCESS] Reflected XSS via event handler
xss_stored_low.py	XSS (Stored)	Low	[SUCCESS] Stored XSS persisted
xss_stored_medium.py	XSS (Stored)	Medium	[SUCCESS] Stored XSS filter bypass
xss_stored_high.py	XSS (Stored)	High	[FAIL] Stored XSS blocked at high
xss_dom_low.py	XSS (DOM)	Low	[SUCCESS] DOM XSS sink (document.write) present
xss_dom_medium.py	XSS (DOM)	Medium	[SUCCESS] DOM XSS at medium — verify client-side
xss_dom_high.py	XSS (DOM)	High	[SUCCESS] DOM XSS at high — verify client-side
weak_session_ids_low.py	Weak Session IDs	Low	[INFO] Session ID page loaded; IDs in cookies
weak_session_ids_medium.py	Weak Session IDs	Medium	[SUCCESS] Session IDs predictable
weak_session_ids_high.py	Weak Session IDs	High	[SUCCESS] High-level session IDs collected
javascript_attacks_low.py	JavaScript Attacks	Low	[SUCCESS] JavaScript token validation bypassed
javascript_attacks_medium.py	JavaScript Attacks	Medium	[FAIL] JS challenge blocked
javascript_attacks_high.py	JavaScript Attacks	High	[FAIL] JS high challenge blocked
open_http_redirect_low.py	Open HTTP Redirect	Low	[INFO] Module not present in DVWA v1.10
open_http_redirect_medium.py	Open HTTP Redirect	Medium	[INFO] Module not present in DVWA v1.10
open_http_redirect_high.py	Open HTTP Redirect	High	[INFO] Module not present in DVWA v1.10

6. Recommendations

The following recommendations are prioritised by risk severity and ease of implementation:

1. Set DVWA security to Impossible in production-like environments to study secure implementations. The Impossible level uses prepared statements and proper input validation throughout.
2. Replace all string concatenation SQL queries with prepared statements and parameterized queries. This is the single most impactful change to eliminate SQL Injection and Blind SQL Injection.
3. Disable `shell_exec`, `exec`, `system`, `passthru`, and similar PHP functions in `php.ini` unless strictly required. Where command execution is necessary, use allowlists and PHP `escapeshellarg/escapeshellcmd`.
4. Deploy Content-Security-Policy headers and implement context-aware output encoding for all user input to eliminate XSS vulnerabilities across all delivery vectors (reflected, stored, DOM).
5. Implement anti-CSRF tokens using a cryptographically secure random value on all state-changing operations, with server-side validation on every request — not just Referer header checking.
6. Store uploaded files outside the web root and validate file content (magic bytes) rather than relying on file extension or MIME type. Rename all uploaded files and disable script execution in upload directories.
7. Use cryptographically random session identifiers with sufficient entropy (at least 128 bits). Implement session regeneration after authentication and appropriate session timeout policies.
8. Implement rate limiting, account lockout policies, and CAPTCHA on all authentication endpoints to prevent automated brute-force attacks regardless of the underlying authentication mechanism.

This assessment was conducted exclusively in an isolated local Docker environment against a deliberately vulnerable application. All findings are expected and educational in nature. No real systems or user data were affected.

— End of Report —